

ProRL Agent: Rollout-as-a-Service for RL Training of Multi-Turn LLM Agents

Hao Zhang*, Mingjie Liu*, Shaokun Zhang*, Songyang Han, Jian Hu, Zhenghui Jin, Yuchi Zhang, Shizhe Diao, Ximing Lu, Binfeng Xu, Zhiding Yu, Jan Kautz, Yi Dong

Abstract

Multi-turn LLM agents are increasingly important for solving complex, interactive tasks, and reinforcement learning (RL) is a key ingredient for improving their long-horizon behavior. However, RL training requires generating large numbers of sandboxed rollout trajectories, and existing infrastructures often couple rollout orchestration with the training loop, making systems hard to migrate and maintain. Under the *rollout-as-a-service* philosophy, we present **ProRL Agent**, a scalable infrastructure that serves the full agentic rollout lifecycle through an API service. **ProRL Agent** also provides standardized and extensible sandbox environments that support diverse agentic tasks in rootless HPC settings. We validate **ProRL Agent** through RL training on software engineering, math, STEM, and coding tasks. **ProRL Agent** is open-sourced at [ProRL Agent](#) and integrated as part of NVIDIA NeMo Gym.

1. Introduction

Recent advances in reinforcement learning from verifiable rewards (RLVR) for large language models (LLMs) are increasingly shifting from single-turn to multi-turn agentic tasks (Cao et al., 2025a; Gao et al., 2025; Guo et al., 2025; Hu et al., 2025; Luo et al., 2025a). Unlike single-turn tasks, multi-turn agentic tasks typically involves interacting with external environments, such as code repositories (Jimenez et al., 2023), web-browser (Zhou et al., 2023), or even full computer operating systems (Xie et al., 2024) via iterative tool use. As a result, they often produce trajectories that often span dozens of turns and tens of thousands of tokens.

Training such agents with RL requires repeatedly rolling out policies in these environments and using the resulting trajectories for optimization. As task scale and complexity grow, rollout generation becomes a major bottleneck due to the heterogeneous environments and non-instantaneous feedback inherent in agentic tasks. For example, a single rollout in software engineering tasks often involves many sequential environment interactions, each of which may incur highly variable latency depending on the execution result or environment response. In response, a number of agentic RL training frameworks have recently emerged (Cao et al., 2025b; Jiang et al., 2025; Liu et al., 2025b; Luo et al., 2025c; Sheng et al., 2025; Tan et al., 2025; Xi et al., 2026).

A counterintuitive design in existing frameworks is the tightly coupling agentic rollout with the RL training stack, with agent lifecycle handled within the trainer. This couples two modules with fundamentally different responsibilities leads to two major limitations.

- Conflicting system requirements:** Rollout and policy training have fundamentally different resource and operational characteristics. Rollout is I/O-intensive, involving sandbox creation, long-lived tool sessions, and asynchronous coordination across hundreds of concurrent instances. Training, by contrast, is GPU-intensive, centered on forward and backward passes, and gradient synchronization. Coupling these workloads causes interference and reduces overall resource efficiency.
- Difficult to migrate and maintain:** When rollout logic is embedded in RL trainer, migrating to a different training backend often requires re-implementing the entire agent execution pipeline. Likewise, improving the rollout infrastructure, such as supporting new runtime environments or tasks, often requires changes that propagate into the training codebase. In practice, this tight coupling slows progress on both fronts,

* Core contribution. © 2026 NVIDIA. All rights reserved.

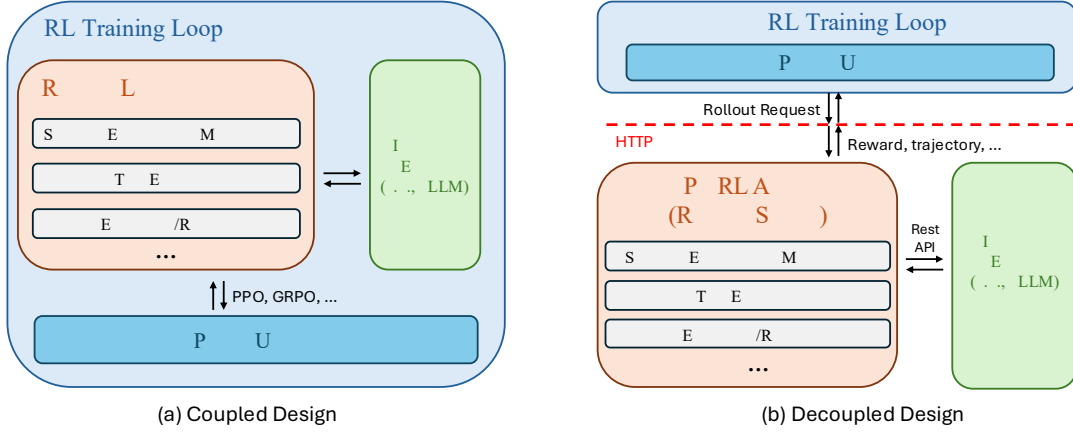


Figure 1: **Coupled vs. decoupled designs.** **Left:** Existing frameworks often embed the full agentic rollout lifecycle inside the RL training stack. **Right:** PRORL AGENT treats rollout as an independent HTTP service. The trainer submits rollout requests and receives completed trajectories and rewards, while the rollout server handles environment execution, tool use, evaluation, and inference coordination. This decoupled design improves resource isolation, portability, and extensibility.

as it makes independent experimentation and optimization on either side more difficult.

These issues are likely to be further exacerbated by the growing need for rapid infrastructure iteration and more effective use of compute resources. If rollout and training are not decoupled from the beginning, the accumulated system complexity can become a serious obstacle to scalability and long-term maintainability.

Drawing inspiration from the inference-as-a-service philosophy adopted by common LLM inference engines (Kwon, 2025; Zheng et al., 2024), we adopt rollout-as-a-service as the core design principle for agentic RL training frameworks, decoupling the trainer from agentic rollout by treating the agentic rollout lifecycle as an independent service. We present PRORL AGENT, an open-source scalable infrastructure for multi-turn agentic rollout in RL training. Instead of implementing rollout as an in-process component of the RL trainer, PRORL AGENT serves the full rollout pipeline, from environment initialization to outcome evaluation, through an HTTP server. This design allows RL trainers to submit task instances and retrieve completed trajectories without managing any part of the rollout lifecycle. On one hand, this decoupled design allows rollout and training to run on different machines, separating I/O-intensive execution from GPU-intensive optimization; on the other hand, it improves extensibility and maintainability by decoupling rollout infrastructure from training backends.

In addition, PRORL AGENT provides several other features that support effective RL training for multi-turn agents. First, it adopts token-in/token-out communication throughout the training pipeline, allowing trainers to directly consume token-level trajectories while avoiding re-tokenization drift (The Agent Lightning (AGL) Team, 2025). This makes training more stable and faithful to the original model outputs. Second, PRORL AGENT provides extensible sandbox environments for agent execution, with flexible support for diverse tools and task. This makes it simple to host heterogeneous agentic tasks within a unified rollout service. Third, PRORL AGENT is designed for rootless deployment in shared cluster environments. This makes it practical to run large-scale agentic rollouts under the permission and isolation constraints common in HPC settings.

We validate PRORL AGENT by integrating it with ProRL training framework (Liu et al., 2025a) for end-to-end RL training on software engineering tasks. Across 4B, 8B, and 14B model scales, it yields strong gains on SWE-Bench Verified. It also performs well in other agentic domains, including MATH, STEM, and coding. ProRL Agent is also integrated as part of NVIDIA NeMo Gym (NVIDIA, 2025).

In summary, the main contributions of this work are:

- We identify the key limitation in existing agentic RL training frameworks: multi-turn agentic rollout is typically tightly coupled with the RL training stack, even though rollout and training have fundamentally different resource and execution characteristics. To address this, we introduce ProRL Agent, an open-source and scalable rollout infrastructure for agent RL training built on the rollout-as-a-service principle, which decouples the full rollout lifecycle from the trainer through a unified HTTP interface.
- We design ProRL Agent with several practical properties for multi-turn RL training, including token-in/token-out trajectory communication to avoid re-tokenization drift, extensible sandboxed environments for heterogeneous tools and tasks, and rootless deployment support for shared HPC clusters.
- We validate ProRL Agent through end-to-end RL training on software engineering tasks with the ProRL training framework. Across 4B, 8B, and 14B model scales, it achieves strong gains on SWE-Bench Verified, while also showing strong performance in other agentic domains such as math, STEM, and coding.

2. Related Work

Frameworks	Training-Rollout Decoupled?	Rootless Sandbox?	Scale-Independent?
SkyRL-Agent (Cao et al., 2025b)	✗	✗	✓
VeRL-Tool (Jiang et al., 2025)	✗	✗	✓
Agent Lightning (Luo et al., 2025c)	✗	✗	✗
rLLM (Tan et al., 2025)	✗	✗	✓
GEM (Liu et al., 2025b)	✗	✗	✓
ProRL Agent (Ours)	✓	✓	✓

Table 1: **Comparison of ProRL Agent with existing frameworks for multi-turn agent RL.** PRORL AGENT decouples rollout from training, supports rootless sandboxing for shared HPC environments, and is independent of any specific training framework.

Multi-turn RL for LLM Agents. Reinforcement learning has been highly effective for improving single-turn reasoning such as mathematics, logic, and coding (Guo et al., 2025; Hu et al., 2025; Shao et al., 2024; Zhang et al., 2026). Building on this progress, recent work has extended RL to multi-turn agentic settings, where agents interact with external environments over long horizons (Cao et al., 2025a; Gao et al., 2025; Jin et al., 2025; Li et al., 2025; Luo et al., 2025a; Wang et al., 2025, 2026). In these settings, a multi-turn agent is naturally formulated as a POMDP (Kaelbling et al., 1998), where agent produces actions through tool calls (Patil et al., 2025; Wang et al., 2024a; Yao et al., 2022; Zhang et al., 2024) and receives environment observations at each step. As tasks become more complex, multi-turn rollouts often span dozens of steps in diverse environments, such as code repositories (Jain et al., 2025; Jimenez et al., 2024), web browsers (Zhou et al., 2023), and even computer operating systems (Xie et al., 2024).

As a result, the infrastructure required to generate, manage, and evaluate these rollouts at scale has become a major bottleneck for RL training. This bottleneck slows both training and the deployment of RL agents. PRORL AGENT is designed to address this challenge by decoupling the full lifecycle of multi-turn agent rollout from the training stack, allowing researchers and practitioners to focus on training algorithms and agent design.

Agent RL Infrastructures. A growing body of work has begun to address the challenges of scalable RL training for agents, including support for diverse tool integration (Jiang et al., 2025; Li et al., 2025), flexible environment abstractions (Liu et al., 2025b; Tan et al., 2025), and efficient rollout scheduling (Cao et al., 2025b). Yet across these frameworks, rollout orchestration, including environment lifecycle management, tool execution, trajectory collection, and evaluation, remains implemented as an in-process library within the training loop. Under this design, adopting a new training backend often requires re-implementing or porting the entire rollout stack. This tight coupling makes rollout infrastructure a major source of friction in multi-turn agent RL, often demanding more engineering effort than the training algorithm itself.

Agentic Sandbox Environments. Multi-turn agent training requires sandboxed environments that provide

isolation, reproducibility, and security at scale. Existing platforms (Jain et al., 2025; Jimenez et al., 2024; Wang et al., 2024b; Yang et al., 2024) have established primary protocols, but they deeply rely on Docker for agent execution. Docker assumes daemon access and root-equivalent privileges, which are often unavailable on shared Slurm-managed HPC clusters. As a result, practitioners often face a trade-off between maintaining separate infrastructure for evaluation and deployment, or incurring the operational complexity of privileged container runtimes on restricted systems. PRORL AGENT addresses this limitation by building its sandbox infrastructure on Singularity, enabling rootless execution and native Slurm integration for large-scale agent training on HPC systems.

3. System Design: Training–Rollout Decoupling

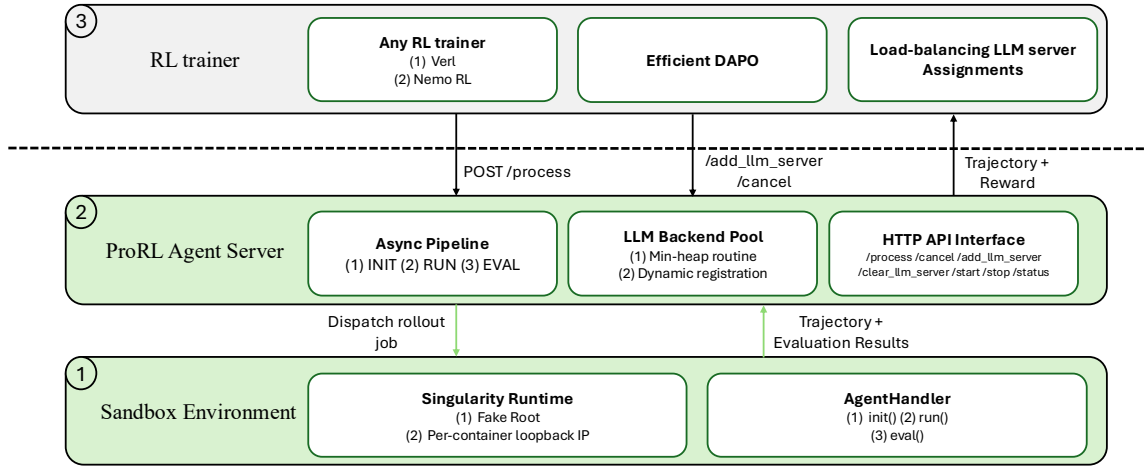


Figure 2: Overview of the PRORL AGENT architecture. The system consists of three components. **(1) Sandbox Environment**: each rollout is executed inside a SingularityRuntime container and orchestrated via AgentHandler, which exposes three lifecycle methods including `init()`, `run()`, and `eval()` for environment setup, multi-turn agent execution, and reward scoring, respectively. **(2) ProRL Agent Server**: an HTTP service that manages rollouts through a three-stage asynchronous pipeline (INIT → RUN → EVAL) with independent worker pools, and maintains a min-heap LLM backend pool supporting dynamic registration and checkpoint swapping. **(3) RL Trainer**: any training framework (e.g., verL, NeMo RL) interacts with the server solely via HTTP, submitting jobs via `POST process` and managing backends via `add_llm_server` and `/cancel`; completed trajectories and rewards are returned to the trainer to update the policy.

3.1. Overview

Training RL agents on agentic tasks normally involves multi-turn interaction with live execution environments, where each data sample spans sandbox environment setup, tool execution, and outcome scoring, a process far more complex than single-step generation. Prior systems typically embed rollout logic directly inside the training loop (Cao et al., 2025b), tightly coupling the agent task loop, execution environment, and RL algorithm. This coupling imposes significant engineering overhead when switching task, and RL trainers.

PRORL AGENT addresses this through a rollout-as-a-service design with **rollout-level decoupling**, in which rollout orchestration is fully separated from the training process. In particular, ProRL Agent Server runs as a standalone HTTP service that accepts a task instance, executes the full agent rollout internally, and returns a completed trajectory with a reward signal. The training framework interacts with the server only through this interface, remaining agnostic to RL infrastructure. This decoupling has three practical consequences.

- The RL trainer and agentic rollout logic can be developed, deployed independently: rollout nodes and

training nodes can be optimized separately for larger throughput.

- Adding a new task requires only implementing a handler plugin on the rollout server side, with no changes to the training code.
- Agentic scaffolds can be modified or replaced without affecting the training infrastructure, as the rollout service and the agent implementation are fully decoupled.

Figure 2 illustrates the overall architecture, which consists of three main components: extensible sandbox environments, the ProRL Agent server for rollout scheduling, and the RL training backend. We introduce each component in turn and describe how they interact within the system.

3.2. Extensible Sandbox Environments

Performing RL training over diverse multi-turn agentic tasks normally requires a sandbox layer that can accommodate heterogeneous task environments and run portably on HPC clusters without privileged access. We build such the sandbox system around two components: a pluggable task abstraction that decouples task-specific logic from the server core, and an HPC-compatible container runtime that enables isolated, rootless agentic tasks execution at scale.

3.2.1. Pluggable Task Abstraction

Different agentic tasks e.g., software engineering, mathematical reasoning, computer use, each require their own environment setup, agent behavior, and reward computation. Hardcoding these differences in the server would make it brittle and always rely on great human efforts. Instead, we encapsulate all task-specific logic in an abstract interface called *AgentHandler*, which defines three core lifecycle methods corresponding to the three pipeline stages:

- `init`: initialize the sandbox environment for the task, configures the agent with corresponding toolset.
- `run`: drives the multi-turn agent loop within the prepared sandbox environment, collecting the action-observation trajectory and any task artifacts.
- `eval`: scores the agent’s output against the ground truth and returns a scalar reward signal for subsequent RL training.

Each handler additionally exposes per-stage error callbacks (`init_exception`, `run_exception`, `eval_exception`) and a `final_result` method for response serialization, ensuring the server always emits a well-formed output even when a rollout fails partway through. Listing 1 illustrates the interface and a minimal registration example.

Listing 1: The *AgentHandler* interface and task registration. Each task domain subclasses *AgentHandler* and registers under a unique name. The server dispatches incoming jobs by matching with different registry.

```
class AgentHandler(ABC):
    @abstractmethod
    async def init(self, job_details) -> (Runtime, Metadata, Config):
        """Provision environment; return (runtime, metadata, config)."""
    @abstractmethod
    async def run(self, job_details) -> dict:
        """Execute agent loop; return trajectory and artifacts."""
    @abstractmethod
    async def eval(self, job_details) -> dict:
        """Score output; return reward signal."""
    # Error callbacks (one per stage) and result serializer
    def init_exception(self, job_details, exc) -> dict: ...
    def run_exception(self, job_details, exc) -> dict: ...
    def eval_exception(self, job_details, exc) -> dict: ...
    def final_result(self, job_details) -> dict: ...
```

When the server receives a job, it reads the task instance, looks up the corresponding handler in the registry, and dispatches to its lifecycle methods in order.

3.2.2. HPC-Compatible Container Runtime

Most agentic sandbox environments assume a cloud or workstation environment where Docker is readily available. HPC clusters, however, typically forbid Docker daemons for security reasons, requiring all user processes to run without root privileges under a batch scheduler such as Slurm. To bridge this gap, we implement *SingularityRuntime*, a container system that requires no persistent daemon and runs entirely as an unprivileged user process to serve sandbox environments.

Container isolation and port management. Each container is launched as a child process in its own session; shutdown proceeds gracefully via `SIGTERM` before escalating to `SIGKILL` if necessary. To support many concurrently running containers on the same node without port conflicts, each container instance is assigned a unique loopback IP address within the `127.x.x.x` range via a thread-safe allocator. Two flags address common HPC constraints: `-fakeroot` grants the container simulated root access for package installation without requiring actual host privileges, and `-network none` optionally disables external network access to isolate rollouts from interference.

Image build pipeline. Container images are packaged as Singularity Image Files (.sif), which encapsulate the full execution environment in a single portable file. This format is particularly well-suited to Slurm shared filesystems, where no persistent container daemon is available. A companion *SingularityRuntimeBuilder* constructs images from Jinja2 templates and supports three caching modes: `SCRATCH` always performs a full rebuild; `VERSIONED` reuses a cached image when the base image and framework version are unchanged; and `LOCK` reuses it whenever the dependency lockfile is identical. The template-driven design enables flexible specialization of runtimes for heterogeneous agentic environments. For example, QEMU-based virtual machines used in GUI-centric tasks can provide custom definition files to the builder without requiring any modifications to the core build logic.

3.2.3. Efficient tool backends

The agent mostly interacts with the environment through tools: it reads and writes files, executes shell commands, runs Python code, and browses the web. Each tool call is a synchronous blocking operation from the agent’s perspective, the agent must wait for the observation before it can decide its next action. Because a typical rollout spans dozens of such calls, per-tool latency compounds directly into total rollout time, and at high concurrency this overhead can dominate LLM inference as the primary bottleneck. We therefore optimize three critical tool backends.

Efficient Bash. Shell execution is the most frequent action across all code-centric agentic tasks. Conventional implementations route bash commands through a `tmux` session, incurring the overhead of terminal multiplexing. We replace this with a `ptyprocess`-based direct pseudo-terminal, which grants the agent a raw shell without the `tmux` intermediary, yielding a significant reduction in shell command round-trip latency.

IPython. When an agent writes and executes Python code across multiple steps, it is often building on its own prior work: importing a library once, then using it repeatedly; defining a helper function, then calling it later. A persistent IPython kernel makes this natural so that variables and imports defined in one step remain available in subsequent steps, so the agent does not need to repeat setup code on every call. The conventional way to host such a kernel is through the Jupyter kernel gateway, but this adds a network round-trip even when the kernel runs on the same machine as the agent. We instead connect to the kernel directly via its in-process API, removing this overhead entirely.

UDS communication. When the agent decides to take an action, such as running a shell command, editing a file, or executing Python, that action is not run directly by the agent process. Instead, it is sent to a small execution server running inside the container, which carries out the action and sends the observation back.

The common transport for this channel is TCP loopback, which works correctly but forces co-located processes that share the same IP to be distinguished only by port numbers, complicating non-conflicting port assignment and it typically offers lower throughput than Unix domain sockets. We replace it with Unix domain sockets (UDS), a simpler IPC mechanism that passes messages through the OS kernel directly without any networking overhead. Since this channel is exercised on every agent action, shaving latency here accumulates meaningfully across a full rollout.

Together, these three optimizations ensure that tool execution does not become the throughput bottleneck as rollout concurrency scales to hundreds of parallel agents.

3.3. ProRL Agent Server

With the sandbox layer handling individual rollout execution, the server's core responsibility during RL training is to orchestrate hundreds of such rollouts concurrently while providing the training framework with live control over the rollout infrastructure.

There are two basic requirements for the server:

- First, the three rollout phases have fundamentally different resource demands: container initialization is I/O-bound, agent execution is LLM-inference-bound, and outcome evaluation ranges from a few milliseconds for direct scoring to several minutes for full test-suite execution. Executing these phases within each job should not limit throughput to the slowest stage.
- Second, the training framework needs dynamic control over LLM inference backends: it must be able to register new servers as the compute cluster scales, swap backends when model checkpoints are updated, and cancel stale in-flight jobs whose gradient batch has already advanced, all without tight coupling to the server internals.

ProRL Agent Server addresses both facets through two mechanisms: **(1) An asynchronous three-stage pipeline** that assigns each rollout phase to an independent worker pool so all three phases can overlap across the job population; and **(2) A lightweight management API** that exposes job submission, per-job cancellation, LLM backend registration, and server lifecycle control to any RL training framework over HTTP. Listing 2 sketches the resulting architecture.

Listing 2: Simplified logic of the ProRL Agent Server. Three independent worker pools drain their respective queues concurrently.

```
# -- Three independent worker pools -----
STAGES = [INIT, RUN, EVAL]
queues = {s: Queue() for s in STAGES}      # thread-safe FIFO per stage
pools = {s: ThreadPool(N[s]) for s in STAGES}
llm_backends = MinHeap()                  # min-heap keyed by in-flight count

def worker_loop(stage):
    while running:
        job = queues[stage].get()
        if job.id in discarded: continue
        with job.timer.phase(stage):      # only this phase counts toward timeout
            try:
                result = handler[stage](job)
            except Exception as e:
                result = handler[stage+' _exception'](job, e)
        job.store(stage, result)
        if stage == RUN:
            cleanup(job.runtime)           # free container before eval starts
```



```

if stage != EVAL:
    queues[next_stage[stage]].put(job)
else:
    job.done.set()                # unblock the waiting HTTP handler

```

3.3.1. Three-Stage Rollout Pipeline

Think of the rollout process as an assembly line. A naive implementation would assign one worker to each job and have that worker do everything: start the container, run the agent, and score the result, before picking up the next job. The problem is that each phase takes a very different amount of time and uses a very different resource. Container startup is slow because it is waiting on disk I/O and the network. Agent execution is fast per call but fires dozens of LLM requests, so it is bottlenecked by GPU throughput. Evaluation can be nearly instant for a math answer check, or take several minutes for a full test suite. A single worker sitting through all three phases in sequence would spend most of its time idle, waiting for whichever phase happens to be slow.

In ProRL Agent server, the solution is to decouple the phases, exactly as a factory decouples assembly stations. The three lifecycle methods of `AgentHandler` (Section 3.2.1) map onto three independent worker pools, each with its own queue. Initialization workers continuously pull new jobs, spin up containers, and hand them off to the rollout queue. Rollout workers drive agent loops and hand completed trajectories to the evaluation queue. Evaluation workers score results and return them to the caller.

At any moment, all three pools are busy on different jobs simultaneously: while one job is being evaluated, a second is mid-rollout, and a third is having its container started. Because the pools are independent, they can also be sized separately to match their respective workloads, with more init workers to absorb the slow I/O startup, or more eval workers when test suites are particularly long.

3.3.2. LLM Backend Management

Listing 3: Simplified logic of the ProRL Agent Server. The management API gives the training framework full control over jobs and LLM backends at runtime.

```

# -- Management API (HTTP endpoints) -----
POST /add_llm_server {"address": "http://host:port/v1"} # register backend
POST /clear_llm_server                                # flush all backends
POST /process {"instance": {...}, "sampling_params": {...}} # submit job
POST /cancel {"job_id": "..."} # abort running job
POST /start | POST /stop # server lifecycle
GET /status # queue depths

```

Every step of the agent loop requires an LLM completion: the model receives the current conversation history and produces the next action. When hundreds of rollouts run in parallel, these calls arrive at the inference layer simultaneously and at high frequency. A single LLM server (e.g., vLLM server) quickly becomes a bottleneck, so RL training typically co-deploys a pool of LLM servers and distributes inference traffic across them. The ProRL Agent Server manages this pool directly, handling both registration and routing so that the training framework does not need to coordinate LLM access itself.

Dynamic registration and checkpoint swapping. LLM backends are registered and deregistered through the management API at any time during a training run. We show the simplified logic in Listing 3.3.2 When a new LLM server comes online, the trainer calls `POST /add_llm_server` with the server's endpoint; the server is immediately available for routing. When the RL trainer updates the policy checkpoint (e.g., after a gradient synchronization step), the old LLM weights are no

Load balancing via min-heap. Each LLM backend is stored alongside an assignment counter in a min-heap. Every time the rollout stage needs to issue an LLM call, ProRL Agent server automatically selects the backend with the lowest counter and assigns that entire task to the selected LLM. The counter is incremented once per task (rather than per call), ensuring that all subsequent calls within the same task are consistently routed to the same backend to maximize prefix cache reuse. After assignment, the backend’s updated counter is used to maintain its position in the heap:

$$s^* = \arg \min_s w_s, \quad w_{s^*} \leftarrow w_{s^*} + 1,$$

where w_s counts the total number of inference calls assigned to server s since it was registered. Because selection is proportional to assignment count, servers that receive heavier traffic fall back in priority, achieving a round-robin-like balance across the pool without requiring any global synchronization. The entire operation is protected by a single lock, making it safe under the high concurrency of the rollout worker pool.

3.3.3. Token-in/Token-out

If trajectories are transmitted through the training pipeline as plain text, re-tokenization on the can be lossy: the resulting token sequence may differ from the one originally generated during rollout (The Agent Lightning (AGL) Team, 2025), leading to unintended off-policy discrepancies.

PRORL AGENT eliminates this *re-tokenization drift* by using token IDs as the canonical representation throughout the entire training process. The rollout worker sends `prompt_ids` directly to the LLM backend and receives `response_ids` with per-token log-probabilities; each message additionally carries `input_ids`, `output_ids`, and `logprobs` fields that are populated at generation time and propagated unchanged. During multi-turn rollouts, prior assistant turns retain their original token IDs and are concatenated directly into the input buffer; only new messages (e.g., environment observations) are tokenized and appended. This ensures that every token ID returned to the trainer is identical to the one produced during rollout.

3.3.4. Job Lifecycle and Cancellation

We then describe the lifecycle of each job instance and the cancellation mechanism, which together provide greater flexibility for RL trainers.

Phase-aware timeouts. Each job is associated with a `PausableTimer` that accumulates elapsed time only during active pipeline stages (*init*, *run*, and *eval*), while excluding time spent waiting in inter-stage queues. This design ensures that the timeout budget reflects actual execution time rather than transient server-side delays.

Cancellation. The training framework can abort any in-flight job at any time via `POST /cancel`. Once received, ProRL Agent server will: (i)mark the job as discarded so that any worker that has not yet dequeued it will skip it; (ii)cancel the currently executing async task; (iii)close the associated container runtime to release resources immediately; and (iv)signal the job’s completion event so the waiting HTTP handler returns without blocking. This enables the RL trainer to discard incomplete rollouts once a sufficient number of valid samples has been collected.

Fault isolation. Each pipeline stage registers a dedicated exception callback. Once failure, the callback populates `JobDetails` with a structured fallback result and sets the completion event, preventing any single failed rollout from stalling the shared worker pool.

Graceful shutdown. Once received `POST /stop`, the server cancels all in-flight jobs, terminates Singularity processes via process-group scanning, drains the worker pools, and exits cleanly, leaving no orphaned containers on the node.

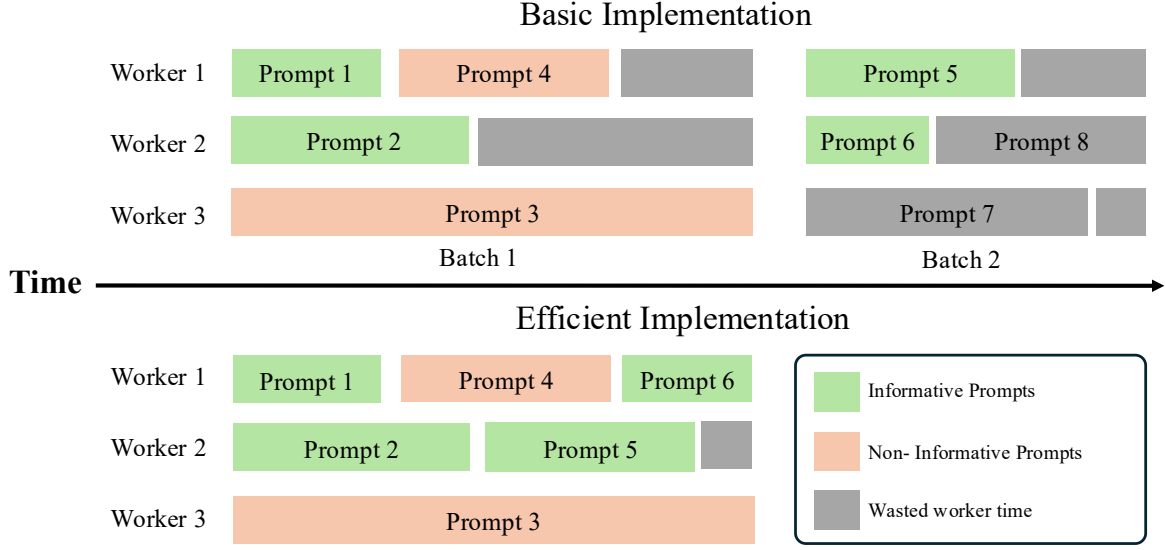


Figure 3: Comparison of DAPO implementations ($n = 4$). Our efficient implementation optimizes worker synchronization, significantly reducing the idle time (waiting period) between rollout generations compared to the baseline batch-by-batch approach.

3.4. Connecting to RL Trainers

Rollout-level decoupling allows the agent server to interface with a wide range of RL trainers. In our implementation, we support both VeRL [Sheng et al. \(2025\)](#) and NeMo RL [nem \(2025\)](#). In addition, we provide several key features that further improve RL training.

Efficient Asynchronous Task Scheduling. On the RL client side, we implement a two-phase hierarchical load balancing strategy that jointly optimizes communication locality and global load balance. In the first phase, LLM servers are assigned preferentially to PRORL AGENT servers on the same physical node, identified through IP address matching, to reduce network latency. In the second phase, any remaining servers are distributed in a round-robin manner to maintain balanced allocation across all available LLM servers.

Efficient DAPO. We adopt Dynamic Sampling Policy Optimization (DAPO) ([Yu et al., 2025](#)) as our core reinforcement learning algorithm. DAPO enhances training stability and data efficiency by filtering out Zero-Variance Prompts—those whose rollouts yield uniform rewards (e.g., all correct or all incorrect) and thus provide no gradient signal. However, applying DAPO to Agent RL is challenging because agent rollouts are typically long-running, asynchronous, and computationally expensive. A naive batch-by-batch implementation—where the trainer requests n prompts, filters out the non-informative ones, and repeatedly triggers new batches until n Informative Prompts are collected—is highly inefficient. This synchronous approach leads to worker idle time and generates redundant rollouts that exceed the target count. Furthermore, discarding incomplete rollouts at the end of a batch results in significant data waste. To address these bottlenecks, we implement an asynchronous replenishment mechanism:

1. **Continuous Throughput:** We replenish the job queue as soon as it empties to maintain maximum rollout throughput.
2. **Early Termination:** We terminate remaining active jobs once the target number of Informative Prompts is reached.
3. **Cross-Iteration Persistence:** Unfinished jobs are carried over to the subsequent iteration to preserve partial progress.

As illustrated in Fig. 3, our optimized implementation significantly reduces worker idle time and improves overall hardware utilization compared to the baseline.

4. Experiments

We next present the experimental results of PRORL AGENT across different tasks. We also perform in-depth investigations to provide a better understanding of our infrastructure.

4.1. Experimental Setup

Unless otherwise specified, we adopt DAPO (Yu et al., 2025) as the default RL algorithm which filters out instances that are either too easy (resolved ratio 100%) or too hard (resolved ratio 0%). We use a batch size of 32, a mini-batch size of 8, and generate 8 rollouts per instance. Rollouts with errors are excluded from gradient computation. The KL coefficient is set to 1×10^{-4} and the learning rate to 1×10^{-6} . All RL training is performed on 32 NVIDIA H100 GPUs.

Table 2: Comparison of performance on SWE-Bench Verified across models of different scales. We report the reproduced performance and, where available, the reported results from prior work. Across all model sizes

Size	Model	Reproduced	Reported
4B	Qwen3-4B-Instruct-2507	14.8	–
	ProRL Agent-4B (RL)	21.2	–
8B	Qwen3-8B	9.6	–
	SkyRL-Agent-8B-v0	–	9.4
	ProRL Agent-8B (RL)	18.0	–
14B	Qwen3-14B	15.4	–
	SkyRL-Agent-14B-v0	–	21.6
	ProRL Agent-14B (RL)	23.6	–

4.2. Main Results on Software Engineering

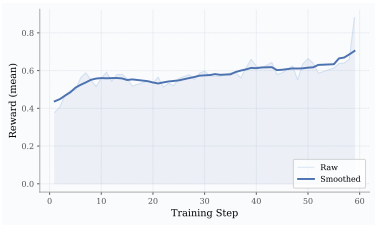
We primarily evaluate PRORL AGENT on software engineering tasks. Specifically, we train Qwen3-4B-Instruct-2507, Qwen3-8B, and Qwen3-14B on the 293-instance subset of SWE-Gym used in SkyRL-v0 (Cao et al., 2025a). For the thinking models, Qwen3-8B and Qwen3-14B, we enable thinking mode during training. The results are reported in Table 2.

As shown in Table 2, PRORL AGENT consistently improves performance across all model sizes. Compared with SkyRL-v0 (Cao et al., 2025a), the gains are particularly notable for the 8B model, where PRORL AGENT achieves nearly a $2\times$ improvement on SWE-Bench Verified. These results suggest that our infrastructure provides a more effective and stable foundation for RL training on software engineering agents.

4.3. Generality Across Agent Domains

Beyond software engineering agents, we further demonstrate the generality of PRORL AGENT by conduct RL training on other domains.

STEM Agent. We further train a STEM agent designed to solve complex question-answering tasks across science, technology, engineering, and mathematics. Its primary tool is *web search*, which enables retrieval of external knowledge for open-domain reasoning. In addition, the agent is equipped with the Bash and IPython tools provided by our infrastructure, allowing it to write and execute code for numerical computation and symbolic problem solving. For the web search backend, we use Tavily. For training data, we follow the ProRL recipe (Liu et al., 2025a) and use the SCP-116K dataset (Lu et al., 2025).



(a) STEM agent.

Table 3: Ablation study of the proposed system components. Action Time denotes the average time required to execute shell-command actions. Each component improves rollout throughput, either by increasing GPU utilization or by reducing action execution time.

Load Balancing	Efficient Bash	Stale Job Cleanup	Action Time (s)	GPU Util (%)	Throughput (instance/sec)
✓	✓	✓	0.42	78	0.37
	✓	✓	0.42	42	0.25
✓		✓	0.78	68	0.29
✓	✓		0.42	65	0.30

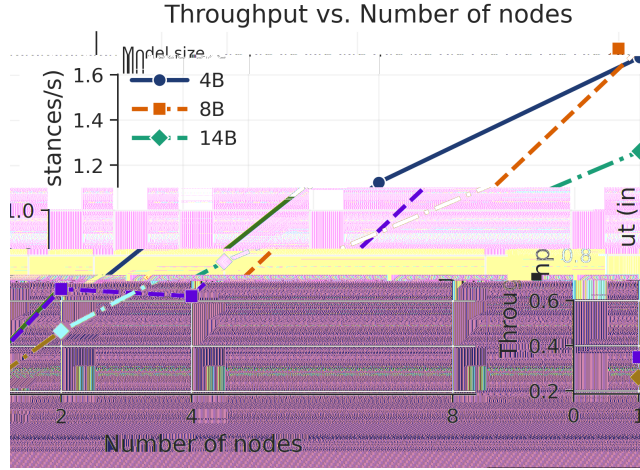


Figure 5: Rollout throughput (instances/sec) on software engineering tasks versus the number of compute nodes. The near-linear increase in throughput demonstrates that ProRL AGENT scales efficiently with additional compute resources.

4.4. System Analysis

4.4.1. Scalability Across Compute Nodes

To evaluate the scalability of ProRL AGENT, we measure rollout throughput (instances per second) on software engineering tasks as the number of compute nodes increases. The results are shown in Fig. 5.

As shown in Fig. 5, throughput increases nearly linearly with the number of nodes, indicating that ProRL AGENT can effectively leverage additional compute resources with minimal scaling overhead. This scalability is particularly valuable for RL training, where efficient rollout generation is often the main system bottleneck and directly affects overall training efficiency.

4.4.2. Component Ablations

We then conducted ablation experiments to evaluate the effectiveness of key components of the ProRL Agent including Load Balancing (LB), Efficient Bash (EB), Stale job Cleanup (SC). Specifically, we measure the rollout throughput of DAPO training on Qwen3-14B-Instruct-2507 using 8 H100 GPUs, with each component removed in turn. For the variant without Load Balancing, we use a simple baseline assignment strategy that distributes an equal number of instances to each LLM server. For the variant without Efficient Bash, we replace our optimized implementation with the original Bash implementation from OpenHands (Wang et al., 2024b). For the variant without Stale Job Cleanup, we wait for all jobs to finish before proceeding. The results in Tab. 3 show that each proposed component contributes to higher rollout throughput during DAPO training. In particular, Load Balancing and Stale Job Cleanup improve throughput by increasing GPU utilization, while Efficient Bash improves throughput by reducing action execution time.

5. Conclusion

In this work, we introduce `PRORL AGENT`, a open-source scalable rollout infrastructure for HPC-native multi-turn agent training. By separating the entire rollout lifecycle from policy training, `PRORL AGENT` improves modularity, scalability, and deployability for agent RL. Experiments across software engineering, STEM, math, and code agents demonstrate effective end-to-end RL training, with strong performance gains across multiple model scales. We release `PRORL AGENT` as open source and as part of NVIDIA NeMo Gym, and leave richer environments and improved cluster-scale robustness to future work.

References

- Nemo rl: A scalable and efficient post-training library. <https://github.com/NVIDIA-NeMo/RL>, 2025. GitHub repository. 10
- Shiyi Cao, Sumanth Hegde, Dacheng Li, Tyler Griggs, Shu Liu, Eric Tang, Jiayi Pan, Xingyao Wang, Akshay Malik, Graham Neubig, Kourosh Hakhmaneshi, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Skyrl-v0: Train real-world long-horizon agents via reinforcement learning. 2025a. 1, 3, 11
- Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Skyrl-agent: Efficient rl training for multi-turn llm agent. *arXiv preprint arXiv:2511.16108*, 2025b. 1, 3, 4
- Jiaxuan Gao, Wei Fu, Minyang Xie, Shusheng Xu, Chuyi He, Zhiyu Mei, Banghua Zhu, and Yi Wu. Beyond ten turns: Unlocking long-horizon agentic search with large-scale asynchronous rl. *arXiv preprint arXiv:2508.07976*, 2025. 1, 3
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 1, 3
- Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025. 1, 3
- Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2e-gym: Procedural environments and hybrid verifiers for scaling open-weights swe agents. *arXiv preprint arXiv:2504.07164*, 2025. 3, 4
- Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, et al. Verl-tool: Towards holistic agentic reinforcement learning with tool use. *arXiv preprint arXiv:2509.01055*, 2025. 1, 3
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023. 1
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. 3, 4
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025. 3
- Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998. 3
- Woosuk Kwon. *vLLM: An Efficient Inference Engine for Large Language Models*. PhD thesis, UC Berkeley, 2025. 2
- Xuefeng Li, Haoyang Zou, and Pengfei Liu. Torl: Scaling tool-integrated rl. *arXiv preprint arXiv:2503.23383*, 2025. 3

- Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong. Prorl: Prolonged reinforcement learning expands reasoning boundaries in large language models. *39th Conference on Neural Information Processing Systems*, 2025a. 2, 11, 12
- Zichen Liu, Anya Sims, Keyu Duan, Changyu Chen, Simon Yu, Xiangxin Zhou, Haotian Xu, Shaopan Xiong, Bo Liu, Chenmian Tan, et al. Gem: A gym for agentic llms. *arXiv preprint arXiv:2510.01051*, 2025b. 1, 3
- Dakuan Lu, Xiaoyu Tan, Rui Xu, Tianchu Yao, Chao Qu, Wei Chu, Yinghui Xu, and Yuan Qi. Sep-116k: A high-quality problem-solution dataset and a generalized pipeline for automated extraction in the higher education science domain, 2025. URL <https://arxiv.org/abs/2501.15587>. 11
- Michael Luo, Naman Jain, Jaskirat Singh, Sijun Tan, Ameen Patel, Qingyang Wu, Alpay Ariyak, Colin Cai, Tarun Venkat, Shang Zhu, Ben Athiwaratkun, Manan Roongta, Ce Zhang, Li Erran Li, Raluca Ada Popa, Koushik Sen, and Ion Stoica. Deepswe: Training a state-of-the-art coding agent by scaling rl. 2025a. 1, 3
- Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Tianjun Zhang, Erran Li, Raluca Ada Popa, and Ion Stoica. DeepScaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2>, 2025b. Notion Blog. 12
- Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K Qiu, and Yuqing Yang. Agent lightning: Train any ai agents with reinforcement learning. *arXiv preprint arXiv:2508.03680*, 2025c. 1, 3
- NVIDIA. Nemo gym: An open source library for scaling reinforcement learning environments for llm. <https://github.com/NVIDIA-NeMo/Gym>, 2025. GitHub repository. 2
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=2GmDdhBdDk>. 3
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. 3
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025. 1, 10
- Sijun Tan, Michael Luo, Colin Cai, Tarun Venkat, Kyle Montgomery, Tianhao Wu, Arnav Balyan, Manan Roongta, Chenguang Wang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. rllm: A framework for post-training language agents. 2025. 1, 3
- The Agent Lightning (AGL) Team. No more retokenization drift: Returning token ids via the openai compatible api matters in agent rl. <https://blog.vllm.ai/2025/10/22/agent-lightning.html>, 2025. 2, 9
- Kangrui Wang, Pingyue Zhang, Zihan Wang, Yaning Gao, Linjie Li, Qineng Wang, Hanyang Chen, Yiping Lu, Zhengyuan Yang, Lijuan Wang, Ranjay Krishna, Jiajun Wu, Li Fei-Fei, Yejin Choi, and Manling Li. VAGEN: Reinforcing world model reasoning for multi-turn VLM agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=xpjWEgf8zi>. 3
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. 2024a. 3

- Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2024b. 4, 13
- Zihan Wang, Chi Gui, Xing Jin, Qineng Wang, Licheng Liu, Kangrui Wang, Shiqi Chen, Linjie Li, Zhengyuan Yang, Pingyue Zhang, Yiping Lu, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. Ragen-v2: Understanding reasoning collapse in multi-turn agent reinforcement learning. 2026. 3
- Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Jiaqi Liu, Honglin Guo, yajie yang, Rui Zheng, Junjie Ye, Jiazheng Zhang, Wenxiang Chen, Wei He, Yiwen Ding, Guanyu Li, Zehui Chen, Zhengyin Du, Xuesong Yao, Yufei Xu, Jiecao Chen, Tao Gui, Zuxuan Wu, Qi Zhang, Xuanjing Huang, and Yu-Gang Jiang. Agentgym-RL: An open-source framework to train LLM agents for long-horizon decision making via multi-turn RL. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=ZgCCDwcGwn>. 1
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024. 1, 3
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024. 4
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2022. 3
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>. 10, 11
- Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels. *arXiv preprint arXiv:2412.01981*, 2024. 12
- Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. Offline training of language model agents with functions as learnable weights. In *Forty-first International Conference on Machine Learning*, 2024. 3
- Shaokun Zhang, Yi Dong, Jieyu Zhang, Jan Kautz, Bryan Catanzaro, Andrew Tao, Qingyun Wu, Zhiding Yu, and Guilin Liu. Nemotron-research-tool-n1: Exploring tool-using language models with reinforced reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=yiE16lWzDj>. 3
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024. 2
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023. 1, 3

A. Appendix

Here, we provide a detailed architectural analysis of existing agent RL infrastructures, accompanied by illustrative diagrams.

PRORL-AGENT-SERVER PROCESS PLACEMENT EXAMPLE

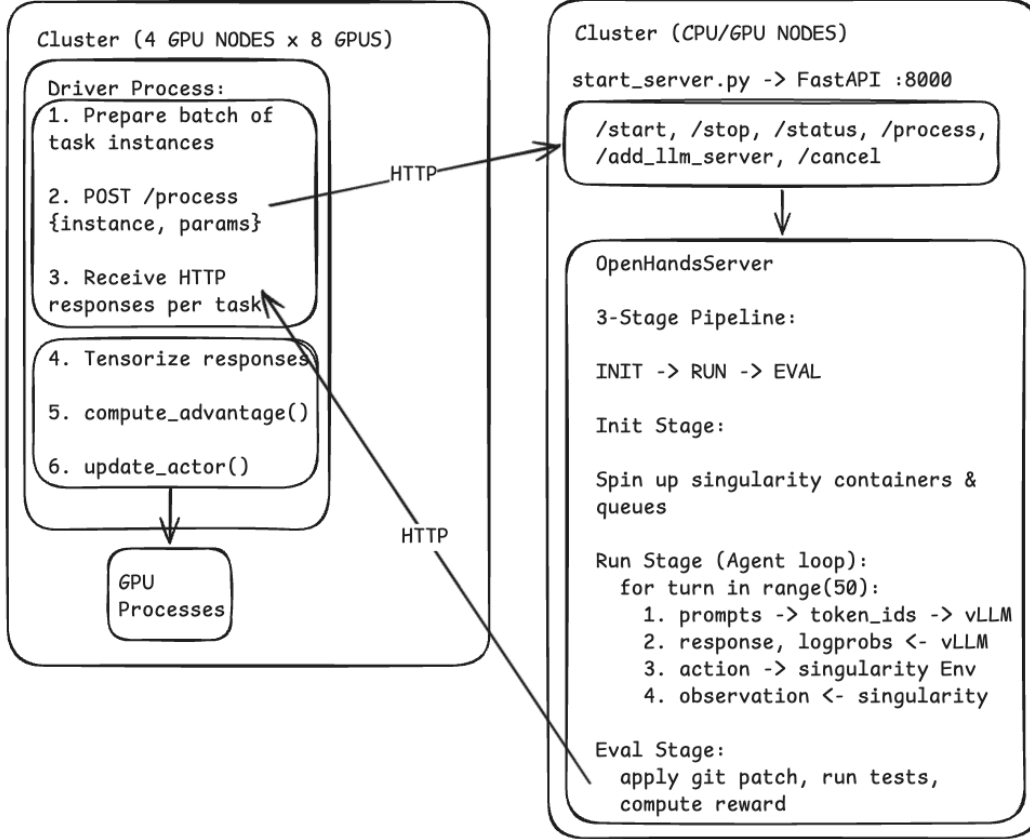


Figure 6: PRORL AGENT separates the full agentic rollout lifecycle, spanning environment management to reward computation, from GPU-intensive training, thereby decoupling I/O-intensive rollout from training.



Figure 8: **Agent Lightning.** Agent Lightning places the training loop, the LightningStoreServer, and all rollout workers within a single process tree. The store runs as a background thread, while rollout workers are spawned as child processes from the trainer. As a result, rollout does not have an independent service lifecycle: if the training process terminates, the store also stops and the rollout workers are disrupted. Thus, rollout remains managed within the training stack rather than being cleanly decoupled.

VeRL-TOOL PROCESS PLACEMENT EXAMPLE

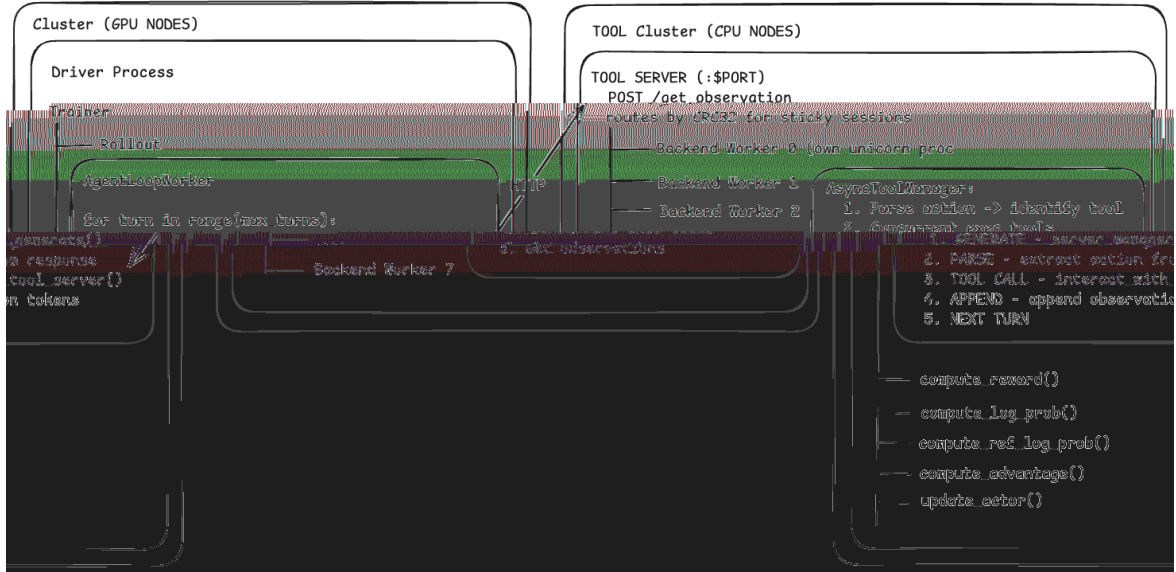


Figure 9: **VeRL-Tool**. VeRL-Tool extends the standard veRL trainer to support multi-turn agent rollouts. The training system manages the agent loop and trajectory collection, while tool execution is offloaded to a separate CPU-based environment service. In this design, rollout control remains inside the trainer.

rLLM PROCESS PLACEMENT EXAMPLE

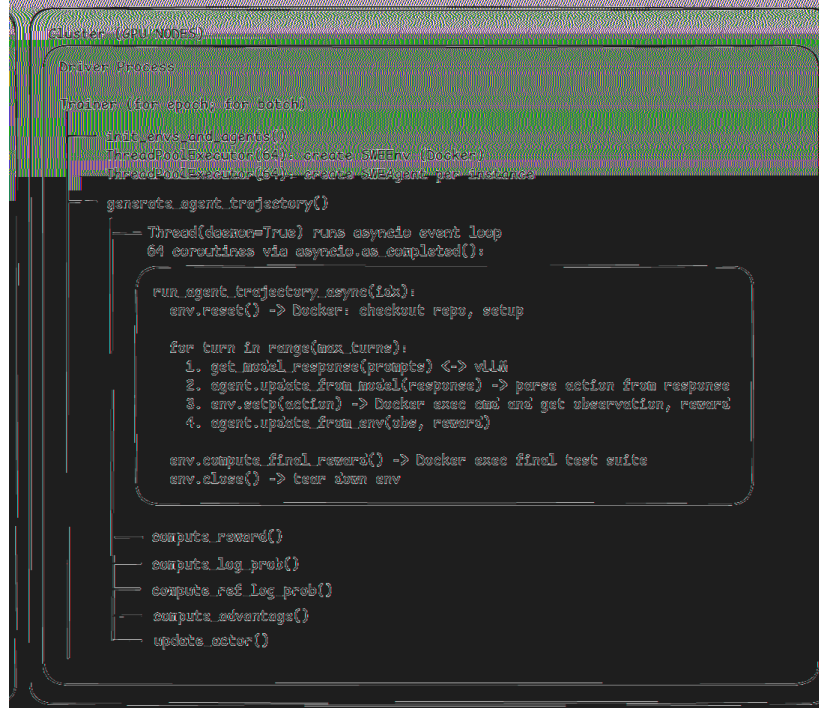


Figure 10: **rLLM: rollout embedded in a monolithic training driver**. rLLM is built on a heavily modified fork of veRL. The agent loop, environment management, and trajectory orchestration all reside within a single driver process. There is no independent rollout service, no persistent trajectory buffer, and no possibility of the rollout surviving independently of the training driver. The full rollout lifecycle remains tightly coupled with the training stack.

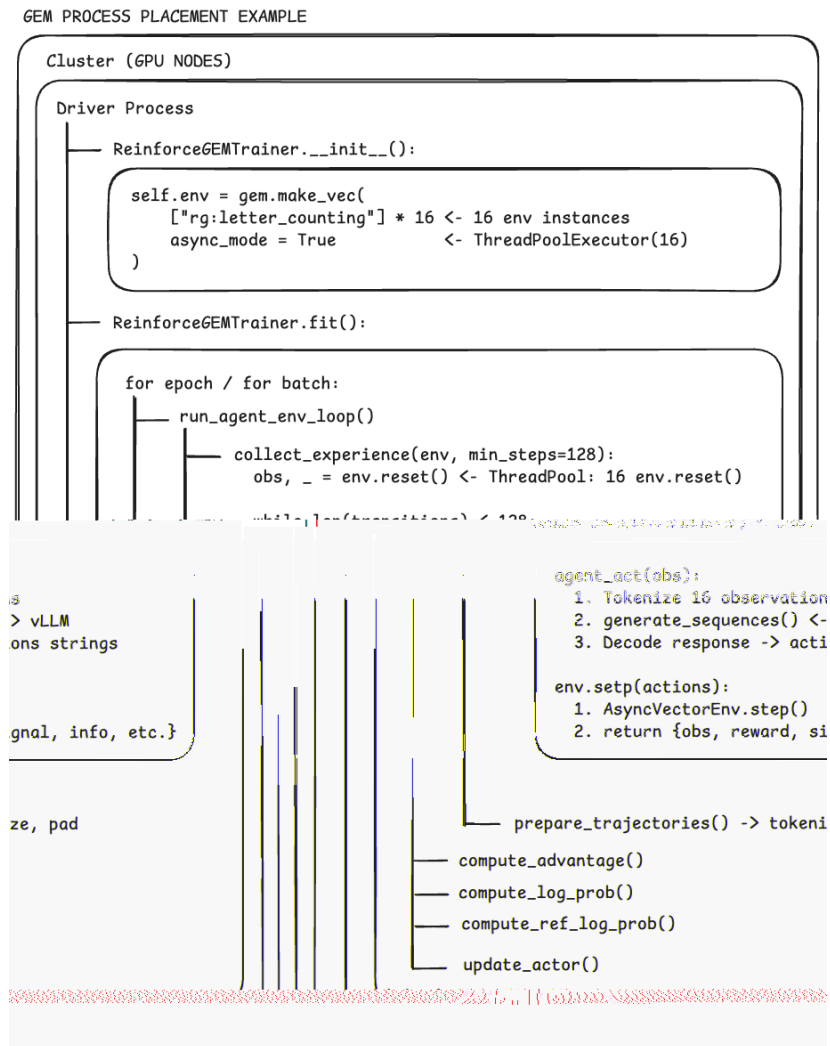


Figure 11: **GEM**. GEM keeps environment execution inside the training process. Environments are instantiated as in-memory Python objects, and environment stepping is performed through direct `env.step()` calls, with parallelism provided only by `ThreadPoolExecutor` threads in the same address space. A single driver process orchestrates both rollout and training, while GPU workers are accessed remotely via Ray RPC. As a result, the environment and rollout lifecycle remain fully embedded in the training stack.